Lesson 2: Key Press Reactor

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: sequences, events, and the game loop. Your job is to build something that reacts when you press a key - and this is also the start of your Semester 1 project, so save it somewhere safe and give it a name you'll recognize.

Tip: this lesson's Assets folder already has a ready-to-use placeholder character image (and an optional sound effect for Level 3) - you don't need to find or make your own art today.

Setup

1. Create a new Blueprint project named [YourName]Game (Blank template).
2. Turn off Lumen - Unreal's dynamic lighting/reflections system, the single biggest reason this engine can feel sluggish on older or integrated-graphics laptops, and something a flat 2D-style scene has no use for anyway: Edit > Project Settings > Engine > Rendering. Under Global Illumination, set Dynamic Global Illumination Method to None. Under Reflections, set Reflection Method to None.
3. In the World Outliner, select the level's default Directional Light and Sky Atmosphere (and Sky Light, if one is present), then delete them - the fully Unlit look from the next step needs no light sources in the level at all.
4. Import this lesson's placeholder sprite into the project: in the Content Browser, drag player_circle_orange.png in from this lesson's Assets folder (or Import). Unreal creates a Texture asset from it automatically - accept the default settings.
5. Create a new Material: Content Browser > Add (+) > Material, name it M_Player_Orange. Open it, select the Material output node itself, and in its Details panel change Shading Model to Unlit. Right-click in empty graph space, search "Texture Sample," add one, and set its Texture to the player_circle_orange asset you just imported. Wire its RGB output into the Material node's Emissive Color input (Unlit materials show only their Emissive Color - that's what makes them render at full brightness with zero light sources). Save and close. This is the real supplied sprite, not a stand-in flat color - Unreal can texture a Material exactly the way every other engine textures a sprite, so that's what this course does here too.
6. Create a new Blueprint Class to serve as your placeholder character: Content Browser > Add (+) > Blueprint Class, parent Static Mesh Actor, name it BP_Player. Open it and select the inherited Static Mesh component already sitting in its Components panel; in its Details panel set Static Mesh to Cube (Engine content) and Material to M_Player_Orange, then under Transform change Mobility from Static to Movable - a Static component won't actually move at runtime no matter what Blueprint logic tells it to,

and setting this on the class itself (rather than a placed instance) means every future instance of BP_Player inherits it automatically.

7. Still inside BP_Player, open its Class Defaults (the Class Defaults button in the toolbar) and, under the Input category, set Auto Receive Input to Player 0. BP_Player is a plain Actor, not a possessed Pawn, so this is what actually lets it receive keyboard input at all - skip it and every Input Action you add later will silently never fire, with no error anywhere to point at why. Compile, Save, then drag an instance of BP_Player into the level.
8. Stop a default flying camera pawn from taking over on Play: Content Browser > Add (+) > Blueprint Class, parent Game Mode Base, name it BP_NoPawnGameMode. Open it and in Class Defaults set Default Pawn Class to None (a raw, non-Blueprint GameMode won't let you edit this field at all - it must be a Blueprint). Compile, Save. Then Window > World Settings > Game Mode section > GameMode Override > BP_NoPawnGameMode.
9. Place Actors panel > search "Camera" > drag a Camera Actor into the level. Position it above BP_Player (raise its Location Z, keep X/Y matching BP_Player) and set its Rotation to Pitch = -90, Yaw = -90, Roll = 0 (looking straight down, oriented so Right reads as screen-right). In its Details panel, set Camera Settings > Projection Mode to Orthographic and Ortho Width to 2000 - a fixed, non-perspective view standing in for the fixed-size stage every other engine in this course uses, instead of Unreal's default open 3D world. Then find Auto Activate for Player and set it to Player 0, so this camera becomes the active view automatically with no Pawn required.
10. In the Content Browser, click Add (+) > Input > Input Action four times, creating IA_MoveRight, IA_MoveLeft, IA_MoveUp, and IA_MoveDown. Leave each one's Value Type as the default (a simple on/off key state, not an axis) - Unreal calls these Input Actions, and they're the current, recommended way to handle input (the older "Action/Axis Mappings" system is deprecated).
11. In the Content Browser, click Add (+) > Input > Input Mapping Context, name it IMC_Default. Open it, click the + next to Mappings four times to add all 4 Input Actions, then under each one click the + to add a key: Right arrow under IA_MoveRight, Left arrow under IA_MoveLeft, Up arrow under IA_MoveUp, Down arrow under IA_MoveDown.
12. Open BP_Player (not the Level Blueprint - everything for this lesson lives inside BP_Player itself) and add an Event BeginPlay node, then chain: Get Player Controller (Index 0) -> Get Enhanced Input Local Player Subsystem -> Add Mapping Context (Context = IMC_Default, Priority = 0). This one-time step is what actually turns your key mappings on - without it, none of the Input Actions below will fire no matter how they're wired. Doing this inside BP_Player itself, rather than the level's own Level Blueprint, is what keeps BP_Player fully self-contained: drop it into a different level later and it still works immediately, with nothing else to remember.

Level 1 - Minimal (MVP)

One key press causes one visible reaction. This is a complete finish for today - reaching this is enough to move on to next week.

13. In BP_Player's own Event Graph, right-click and search for IA_MoveRight to add an Enhanced Input Action Event node for it.
14. Add an "Add Actor World Offset" node. Leave its Target pin unconnected - inside BP_Player's own graph, an unwired Target automatically means Self, so there's no

separate reference to drag in the way a Level Blueprint would need. Connect the Enhanced Input Action Event's Triggered pin (fires every frame for as long as the key is held down, not just once) into the Add Actor World Offset node's execution input. Right-click the Delta Location pin and choose "Split Struct Pin" - it starts collapsed as a single Vector pin, and you can't wire a value into just one axis until it's split into separate X/Y/Z pins. Add a "Get World Delta Seconds" node, multiply it by a Speed value (300 is a reasonable start), and feed that into Delta Location's X - scaling by delta time is what makes BP_Player glide smoothly instead of jumping a fixed amount, the same idea as Unity's Time.deltaTime or Godot's delta.

15. Click Compile, then Play to test - click inside the Play viewport first so it actually has keyboard focus, then hold (not just tap) the Right arrow and confirm BP_Player glides for as long as it's held. Once you're done testing, press Shift+F1 to release the viewport's mouse capture and get your cursor back for editing - Unreal shows this same "Shift+F1 for Mouse Cursor" hint in the corner of the Play viewport as a reminder.

Alternate Path: C++ (code, optional advanced track)

If you'd rather write code than wire Blueprint nodes, you can build the same movement in C++, reusing the same IA_MoveRight and IMC_Default assets from Setup. This needs a code editor (Visual Studio on Windows, or your platform's equivalent) hooked up to Unreal - ask if you're not sure yours is set up.

16. In [YourName]Game.Build.cs, add "EnhancedInput" to PublicDependencyModuleNames (alongside the existing Core/CoreUObject/Engine/InputCore entries) - this is a one-time project setting that makes the Enhanced Input C++ classes available to your code.
17. Tools > New C++ Class, choose Actor as the parent class, name it MovableActor, click Create Class (Unreal compiles and opens your code editor).
18. In MovableActor.h, declare an override of SetupPlayerInputComponent, a MoveRight() function, and two UPROPERTY pointers - one for the Input Action, one for the Mapping Context - so you can assign IA_MoveRight and IMC_Default from the Details panel (see code below).
19. In MovableActor.cpp's BeginPlay(), get the first player controller's Enhanced Input Local Player Subsystem and call AddMappingContext() - the C++ equivalent of the Blueprint path's Add Mapping Context node.
20. Implement SetupPlayerInputComponent() to cast PlayerInputComponent to UEnhancedInputComponent and bind the MoveRightAction to your MoveRight() function on the Started trigger event.
21. Implement MoveRight() to call AddActorWorldOffset() - the same underlying function the Blueprint path's node calls.
22. Compile (the Compile button in the editor toolbar), drag a MovableActor into the level in place of BP_Player, and in its Details panel assign MoveRightAction to IA_MoveRight and MappingContext to IMC_Default.
23. Play, and test with the Right arrow key.

```
// MovableActor.h
virtual void SetupPlayerInputComponent(UInputComponent*
PlayerInputComponent) override;
void MoveRight();

UPROPERTY(EditAnywhere, Category="Input")
class UInputAction* MoveRightAction;
```

```
UPROPERTY(EditAnywhere, Category="Input")
class UInputMappingContext* MappingContext;

// MovableActor.cpp
void AMovableActor::BeginPlay() {
    Super::BeginPlay();
    if (APlayerController* PC = GetWorld()->GetFirstPlayerController()) {
        if (UEnhancedInputLocalPlayerSubsystem* Subsystem =

ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(PC-
>GetLocalPlayer())) {
            Subsystem->AddMappingContext(MappingContext, 0);
        }
    }
}
void AMovableActor::SetupPlayerInputComponent(UInputComponent*
PlayerInputComponent) {
    if (UEnhancedInputComponent* EnhancedInput =
Cast<UEnhancedInputComponent>(PlayerInputComponent)) {
        EnhancedInput->BindAction(MoveRightAction,
ETriggerEvent::Triggered, this, &AMovableActor::MoveRight);
    }
}
void AMovableActor::MoveRight() {
    float Speed = 300.f;
    AddActorWorldOffset(FVector(Speed * GetWorld()->GetDeltaSeconds(),
0.f, 0.f));
}
```

This mirrors the Blueprint path's Add Actor World Offset node - same underlying engine call, just from code, and the same IA_MoveRight/IMC_Default assets either path can use interchangeably. Binding to ETriggerEvent::Triggered instead of Started, and scaling by GetDeltaSeconds(), is what makes this glide smoothly while held instead of stepping once per press - the same continuous-hold behavior as the Blueprint path. Level 2/3: add MoveLeft/MoveUp/MoveDown the same way, each with its own UInputAction property bound to the matching Input Action asset from Setup. Level 3's five goals are all reachable from C++ too: SetActorHiddenInGame() for the blink timer, UGameplayStatics::PlaySound2D() for sound, SetMaterial() on the mesh component for recolor, and FMath::Clamp() for the boundary - same ideas as the Blueprint path, just called from code.*

Level 2 - Complete (Alpha/Beta)

All 4 arrow keys move your character in 4 directions.

- Add the other 3 directions the same way: right-click and add an Enhanced Input Action Event for IA_MoveLeft, IA_MoveUp, and IA_MoveDown (already mapped to their keys in IMC_Default from Setup), each with its own Add Actor World Offset node - Target left unwired (defaults to Self, same as Right), Triggered pin (not Started) driving it, Delta Location scaled by Get World Delta Seconds x Speed the same way as Right, moving in the matching direction (Left = -X, Up = +Y, Down = -Y, matching the Camera's Yaw from Setup).

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Animate: make BP_Player blink (toggle hidden/visible) while moving, and stay visible when idle - drive it from Event Tick with a timer and Set Actor Hidden In Game, kept separate from the per-direction Enhanced Input Action Events.
- Sound: chain a Play Sound 2D node (using an imported sfx_blip.wav Sound Wave asset) off the Triggered pin, gated by a short SoundCooldown timer (count it down by Get World Delta Seconds, only play and reset it back to ~0.15 when it's <= 0) - Play Sound 2D is fire-and-forget with no way to check whether it's already playing, so this cooldown is what turns it into a repeating beep-beep for as long as the key is held, instead of one blip per press, matching the other engines' continuous-while-held sound.
- Recolor: import the other 4 placeholder circles (player_circle_red.png, player_circle_blue.png, player_circle_green.png, player_circle_yellow.png, all in this lesson's Assets folder) the same way Setup imported the orange one, then build one Material per color the same way Setup built M_Player_Orange - a Texture Sample from that color's Texture wired straight into Emissive Color, Shading Model = Unlit. Swap which Material is applied to BP_Player with a Set Material node per direction (and back to M_Player_Orange when idle) - the same real-sprite-swap every other engine's Recolor goal uses, just with an Unreal Material standing in front of the PNG instead of the PNG being assigned directly. If Set Material doesn't show up when you search for it, see this lesson's trouble section - it's a known Blueprint search quirk, not a missing feature.
- Clamp: keep BP_Player inside a defined play area using Get Actor Location, Clamp (Float) on X and Y, then Set Actor Location.
- Structure: keep the per-direction pieces (movement, sound, recolor) chained off each direction's own Enhanced Input Action Event, and keep the continuous blink timer in its own Event Tick - don't mix the two execution contexts.

If Something's Wrong

- Nothing happens, no errors anywhere: this is almost always Auto Receive Input, not the Target pin - open BP_Player's Class Defaults and confirm Auto Receive Input is actually set to Player 0. A plain Actor Blueprint never receives keyboard input without this, and there's no error of any kind when it's missing, just an Enhanced Input Action Event that silently never fires.
- Nothing happens: confirm you clicked Compile after editing, and that the white execution wire actually connects your input node to the movement node.
- Nothing happens, even though everything looks wired correctly: click once inside the Play viewport before pressing any keys. With no Pawn being possessed (by design, from Setup), the window doesn't automatically grab keyboard focus the way it used to - a plain click gives it focus.
- Still nothing happens, even after checking Auto Receive Input, wiring, and viewport focus: this is almost always a missing or misconfigured Add Mapping Context call - confirm BP_Player's own BeginPlay actually calls it with IMC_Default, and that IMC_Default itself actually has all 4 keys mapped (open it and check).
- World Settings isn't on the main toolbar: it's under the Window menu instead - Window > World Settings.
- GameMode Override is set, but Default Pawn Class still shows DefaultPawn and won't change: this happens when GameMode Override is pointed at the raw "GameModeBase" entry instead of the BP_NoPawnGameMode Blueprint from Setup - a

non-Blueprint GameMode's fields display as read-only in World Settings. Re-check GameMode Override is actually set to your BP_NoPawnGameMode asset.

- BP_Player moves, but Right moves it up the screen (or some other wrong direction) instead of right: adjust the Camera Actor's Yaw in 90-degree steps (try -90, 90, or 180) until Right visually reads as right - the exact value depends on how the camera ended up facing, and it's easiest to just try values and watch the result.
- BP_Player (or the whole scene) renders solid black: an Unlit material with nothing wired into Emissive Color renders pure black regardless of lighting - open M_Player_Orange and confirm the Texture Sample's RGB output is actually connected to Emissive Color, and that a Texture is actually assigned on the Texture Sample node itself (an empty Texture Sample defaults to black too).
- Movement still happens once per press instead of gliding while held: double check the Enhanced Input Action Event's Triggered pin is driving Add Actor World Offset, not the Started pin - Started only fires the instant a key goes down.
- Can't wire Get World Delta Seconds (or anything else) into just the X or Y of Delta Location - it only shows one combined Vector pin: right-click that pin and choose "Split Struct Pin" to break it into separate X/Y/Z pins first.
- Mouse cursor seems stuck or trapped inside the Play viewport: press Shift+F1 to release it back to the editor - the click that gives the viewport keyboard focus also captures the mouse.
- Searching for "Set Material" doesn't show the right node (or shows something unrelated like "Set Material Info"): this is a known Blueprint search quirk, not a missing feature - either call "Get Static Mesh Component" first (Target left unwired, defaults to Self same as everything else this lesson) and search for Set Material from that component's pin, or uncheck the "Context Sensitive" box in the top-right of the search popup to show every node regardless of what you dragged from.
- Nothing moves, no errors, everything else checks out: select BP_Player's instance in the level and check Mobility under its Transform section. It should already read Movable, inherited from the class default set in Setup - if it somehow shows Static, that's worth fixing directly on BP_Player's own Static Mesh component in the Blueprint editor (not just the one instance), so every instance stays correct.
- Unreal is still the most hardware-demanding engine on our list even with Lumen off - if your laptop struggles, it's completely fine to observe this week's demo and build your Key Press Reactor in a lighter engine instead.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal